Experiment 1: Library Management System

1. Aim

To design and develop a web-based Library Management System that lets the librarian (admin) manage books, issue and return books, calculate fines and share study material, and lets students register, search books, view their borrow history, check fines and download study material.

2. Technologies Used

Component	Technology
Front end	HTML5, CSS3, Jinja2 templates (responsive layout)
Back end	Python 3 with the Flask web framework
Database	SQLite (file based, no server required)
Security	Werkzeug password hashing, session based login, role check on every page
Editor	Visual Studio Code

3. Algorithm

3.1 Issue a book

1. Admin selects a student and a book.
2. If available_copies of the book is 0, show an error and stop.
3. If the student already holds the same book, show an error and stop.
4. Insert a row in issues with issue_date = today and due_date = today + 14 days.
5. Reduce available_copies of the book by 1.

3.2 Return a book and calculate fine

1. Admin clicks Return for an issued book.
2. late_days = return_date - due_date (0 if negative).
3. fine = late_days x 2.
4. Save return_date and fine, increase available_copies by 1.
5. For books not yet returned, the fine shown is calculated using today's date.

4. Source Code

4.1 app.py

```
import os
import sqlite3
from datetime import date, datetime, timedelta
from functools import wraps

from flask import (Flask, flash, g, redirect, render_template, request,
                   send_from_directory, session, url_for)
from werkzeug.security import check_password_hash, generate_password_hash
from werkzeug.utils import secure_filename

app = Flask(__name__)
app.secret_key = "change-this-secret-key"

BASE = os.path.dirname(os.path.abspath(__file__))
DB = os.path.join(BASE, "library.db")
UPLOADS = os.path.join(BASE, "uploads")
os.makedirs(UPLOADS, exist_ok=True)

LOAN_DAYS = 14      # a book can be kept for 14 days
FINE_PER_DAY = 2    # Rs. 2 per day after the due date



# ---------------------------------------------------------------- database
def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB)
        g.db.row_factory = sqlite3.Row
    return g.db



@app.teardown_appcontext
def close_db(_):
    db = g.pop("db", None)
    if db is not None:
        db.close()



def init_db():
    db = sqlite3.connect(DB)
    db.executescript("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        username TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        email TEXT,
        phone TEXT,
        role TEXT NOT NULL DEFAULT 'user'
    );
    CREATE TABLE IF NOT EXISTS books(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        isbn TEXT,
        category TEXT,
        total_copies INTEGER NOT NULL DEFAULT 1,
        available_copies INTEGER NOT NULL DEFAULT 1
    );
    CREATE TABLE IF NOT EXISTS issues(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        book_id INTEGER NOT NULL REFERENCES books(id),
        user_id INTEGER NOT NULL REFERENCES users(id),
        issue_date TEXT NOT NULL,
        due_date TEXT NOT NULL,
        return_date TEXT,
        fine INTEGER NOT NULL DEFAULT 0
    );
    CREATE TABLE IF NOT EXISTS messages(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT, email TEXT, message TEXT, sent_on TEXT
    );
```

```
    CREATE TABLE IF NOT EXISTS materials(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        subject TEXT,
        year INTEGER NOT NULL,
        semester INTEGER NOT NULL,
        filename TEXT NOT NULL,
        uploaded_on TEXT NOT NULL
    );
    """)
    if not db.execute("SELECT 1 FROM users WHERE role='admin'").fetchone():
        db.execute("INSERT INTO users(name,username,password,role) VALUES(?,?,?,?)",
                   ("Librarian", "admin", generate_password_hash("admin123"), "admin"))
    if not db.execute("SELECT 1 FROM books").fetchone():
        samples = [
            ("Introduction to Algorithms", "Cormen", "9780262033848", "Computer Science", 3),
            ("Database System Concepts", "Silberschatz", "9780078022159", "Computer Science", 2),
            ("Engineering Mathematics", "B.S. Grewal", "9788174091215", "Mathematics", 4),
            ("Wings of Fire", "A.P.J. Abdul Kalam", "9788173711466", "Biography", 2),
        ]
        for t, a, i, c, n in samples:
            db.execute("INSERT INTO books(title,author,isbn,category,total_copies,available_copies)"
                       " VALUES(?,?,?,?,?,?)", (t, a, i, c, n, n))
    db.commit()
    db.close()



# ---------------------------------------------------------------------- helpers
def login_required(role=None):
    def deco(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            if "uid" not in session:
                flash("Please log in first.", "error")
                return redirect(url_for("login"))
            if role and session.get("role") != role:
                flash("You are not allowed to open that page.", "error")
                return redirect(url_for("home"))
            return f(*args, **kwargs)
        return wrapper
    return deco



def calc_fine(due_date, return_date=None):
    """Fine = days late x FINE_PER_DAY. Uses today's date if not yet returned."""
    due = datetime.strptime(due_date, "%Y-%m-%d").date()
    end = datetime.strptime(return_date, "%Y-%m-%d").date() if return_date else date.today()
    return max(0, (end - due).days) * FINE_PER_DAY



def with_fines(rows):
    out = []
    for r in rows:
        d = dict(r)
        d["current_fine"] = d["fine"] if d["return_date"] else calc_fine(d["due_date"])
        out.append(d)
    return out



ISSUE_QUERY = """
    SELECT i.*, b.title, b.author, u.name AS student, u.username
    FROM issues i
    JOIN books b ON b.id = i.book_id
    JOIN users u ON u.id = i.user_id
"""



# ---------------------------------------------------------------------- auth
@app.route("/")
def home():
    if session.get("role") == "admin":
        return redirect(url_for("admin_dashboard"))
    if session.get("role") == "user":
        return redirect(url_for("dashboard"))
    return redirect(url_for("login"))
```

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        u = get_db().execute("SELECT * FROM users WHERE username=?",
                             (request.form["username"].strip(),)).fetchone()
        if u and check_password_hash(u["password"], request.form["password"]):
            session.clear()
            session.update(uid=u["id"], name=u["name"], role=u["role"])
            return redirect(url_for("home"))
        flash("Invalid username or password.", "error")
    return render_template("login.html")



@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        f = request.form
        try:
            get_db().execute(
                "INSERT INTO users(name,username,password,email,phone,role) VALUES(?,?,?,?,?,'user')",
                (f["name"].strip(), f["username"].strip(),
                 generate_password_hash(f["password"]), f["email"].strip(), f["phone"].strip()))
            get_db().commit()
            flash("Registration successful. Please log in.", "success")
            return redirect(url_for("login"))
        except sqlite3.IntegrityError:
            flash("That username is already taken.", "error")
    return render_template("register.html")



@app.route("/contact", methods=["GET", "POST"])
def contact():
    if request.method == "POST":
        f = request.form
        get_db().execute("INSERT INTO messages(name,email,message,sent_on) VALUES(?,?,?,?)",
                         (f["name"].strip(), f["email"].strip(), f["message"].strip(),
date.today().isoformat()))
        get_db().commit()
        flash("Thank you! Your message has been sent to the library.", "success")
        return redirect(url_for("contact"))
    return render_template("contact.html")



@app.route("/admin/messages")
@login_required("admin")
def admin_messages():
    msgs = get_db().execute("SELECT * FROM messages ORDER BY id DESC").fetchall()
    return render_template("admin_messages.html", messages=msgs)



@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))



# ------------------------------------------------------------------- admin
@app.route("/admin")
@login_required("admin")
def admin_dashboard():
    db = get_db()
    stats = {
        "books": db.execute("SELECT COALESCE(SUM(total_copies),0) FROM books").fetchone()[0],
        "available": db.execute("SELECT COALESCE(SUM(available_copies),0) FROM books").fetchone()[0],
        "students": db.execute("SELECT COUNT(*) FROM users WHERE role='user'").fetchone()[0],
        "issued": db.execute("SELECT COUNT(*) FROM issues WHERE return_date IS NULL").fetchone()[0],
    }
    return render_template("admin_dashboard.html", stats=stats)



@app.route("/admin/books")
@login_required("admin")
def admin_books():
    q = request.args.get("q", "").strip()
```

```
    like = f"%{q}%"
    books = get_db().execute(
        "SELECT * FROM books WHERE title LIKE ? OR author LIKE ? OR isbn LIKE ? OR category LIKE ?
ORDER BY title",
        (like, like, like, like)).fetchall()
    return render_template("admin_books.html", books=books, q=q)



@app.route("/admin/books/add", methods=["GET", "POST"])
@login_required("admin")
def add_book():
    if request.method == "POST":
        f = request.form
        n = int(f["copies"])
        get_db().execute(
            "INSERT INTO books(title,author,isbn,category,total_copies,available_copies)
VALUES(?,?,?,?,?,?)",
            (f["title"].strip(), f["author"].strip(), f["isbn"].strip(), f["category"].strip(), n, n))
        get_db().commit()
        flash("Book added.", "success")
        return redirect(url_for("admin_books"))
    return render_template("book_form.html", book=None)



@app.route("/admin/books/<int:bid>/edit", methods=["GET", "POST"])
@login_required("admin")
def edit_book(bid):
    db = get_db()
    book = db.execute("SELECT * FROM books WHERE id=?", (bid,)).fetchone()
    if not book:
        flash("Book not found.", "error")
        return redirect(url_for("admin_books"))
    if request.method == "POST":
        f = request.form
        new_total = int(f["copies"])
        issued = book["total_copies"] - book["available_copies"]
        if new_total < issued:
            flash(f"{issued} copies are currently issued; total cannot be lower.", "error")
            return render_template("book_form.html", book=book)
        db.execute("UPDATE books SET
title=?,author=?,isbn=?,category=?,total_copies=?,available_copies=? WHERE id=?",
                   (f["title"].strip(), f["author"].strip(), f["isbn"].strip(),
                    f["category"].strip(), new_total, new_total - issued, bid))
        db.commit()
        flash("Book updated.", "success")
        return redirect(url_for("admin_books"))
    return render_template("book_form.html", book=book)



@app.route("/admin/books/<int:bid>/delete", methods=["POST"])
@login_required("admin")
def delete_book(bid):
    db = get_db()
    if db.execute("SELECT 1 FROM issues WHERE book_id=? AND return_date IS NULL", (bid,)).fetchone():
        flash("Cannot delete: copies of this book are still issued.", "error")
    else:
        db.execute("DELETE FROM issues WHERE book_id=?", (bid,))
        db.execute("DELETE FROM books WHERE id=?", (bid,))
        db.commit()
        flash("Book deleted.", "success")
    return redirect(url_for("admin_books"))



@app.route("/admin/issue", methods=["GET", "POST"])
@login_required("admin")
def issue_book():
    db = get_db()
    if request.method == "POST":
        bid, uid = int(request.form["book_id"]), int(request.form["user_id"])
        book = db.execute("SELECT * FROM books WHERE id=?", (bid,)).fetchone()
        if not book or book["available_copies"] < 1:
            flash("No copies available for this book.", "error")
        elif db.execute("SELECT 1 FROM issues WHERE book_id=? AND user_id=? AND return_date IS NULL",
                        (bid, uid)).fetchone():
            flash("This student already has this book.", "error")
        else:
            today = date.today()
```

```
            db.execute("INSERT INTO issues(book_id,user_id,issue_date,due_date) VALUES(?,?,?,?)",
                       (bid, uid, today.isoformat(), (today + timedelta(days=LOAN_DAYS)).isoformat()))
            db.execute("UPDATE books SET available_copies = available_copies - 1 WHERE id=?", (bid,))
            db.commit()
            flash(f"Book issued. Due date: {(today + timedelta(days=LOAN_DAYS)).isoformat()}",
"success")
            return redirect(url_for("admin_issues"))
    books = db.execute("SELECT * FROM books WHERE available_copies > 0 ORDER BY title").fetchall()
    students = db.execute("SELECT * FROM users WHERE role='user' ORDER BY name").fetchall()
    return render_template("issue_book.html", books=books, students=students)



@app.route("/admin/issues")
@login_required("admin")
def admin_issues():
    show = request.args.get("show", "all")
    sql = ISSUE_QUERY
    if show == "pending":
        sql += " WHERE i.return_date IS NULL"
    elif show == "returned":
        sql += " WHERE i.return_date IS NOT NULL"
    rows = get_db().execute(sql + " ORDER BY i.id DESC").fetchall()
    return render_template("admin_issues.html", issues=with_fines(rows), show=show)



@app.route("/admin/return/<int:iid>", methods=["POST"])
@login_required("admin")
def return_book(iid):
    db = get_db()
    row = db.execute("SELECT * FROM issues WHERE id=?", (iid,)).fetchone()
    if row and not row["return_date"]:
        today = date.today().isoformat()
        fine = calc_fine(row["due_date"], today)
        db.execute("UPDATE issues SET return_date=?, fine=? WHERE id=?", (today, fine, iid))
        db.execute("UPDATE books SET available_copies = available_copies + 1 WHERE id=?",
(row["book_id"],))
        db.commit()
        flash(f"Book returned. Fine: Rs. {fine}", "success" if fine == 0 else "error")
    return redirect(url_for("admin_issues"))



@app.route("/admin/fines")
@login_required("admin")
def admin_fines():
    rows = with_fines(get_db().execute(ISSUE_QUERY + " ORDER BY i.id DESC").fetchall())
    rows = [r for r in rows if r["current_fine"] > 0]
    return render_template("admin_fines.html", issues=rows,
                           total=sum(r["current_fine"] for r in rows), rate=FINE_PER_DAY)



@app.route("/admin/materials", methods=["GET", "POST"])
@login_required("admin")
def admin_materials():
    db = get_db()
    if request.method == "POST":
        file = request.files.get("file")
        if not file or not file.filename:
            flash("Choose a file to upload.", "error")
        else:
            stamp = datetime.now().strftime("%Y%m%d%H%M%S_")
            fname = stamp + secure_filename(file.filename)
            file.save(os.path.join(UPLOADS, fname))
            db.execute("INSERT INTO materials(title,subject,year,semester,filename,uploaded_on)
VALUES(?,?,?,?,?,?)",
                       (request.form["title"].strip(), request.form["subject"].strip(),
                        int(request.form["year"]), int(request.form["semester"]),
                        fname, date.today().isoformat()))
            db.commit()
            flash("Material uploaded.", "success")
        return redirect(url_for("admin_materials"))
    mats = db.execute("SELECT * FROM materials ORDER BY year, semester, title").fetchall()
    return render_template("admin_materials.html", materials=mats)



@app.route("/admin/materials/<int:mid>/delete", methods=["POST"])
@login_required("admin")
```

```
def delete_material(mid):
    db = get_db()
    m = db.execute("SELECT * FROM materials WHERE id=?", (mid,)).fetchone()
    if m:
        path = os.path.join(UPLOADS, m["filename"])
        if os.path.exists(path):
            os.remove(path)
        db.execute("DELETE FROM materials WHERE id=?", (mid,))
        db.commit()
        flash("Material deleted.", "success")
    return redirect(url_for("admin_materials"))



# ---------------------------------------------------------------------- user
@app.route("/dashboard")
@login_required("user")
def dashboard():
    db = get_db()
    uid = session["uid"]
    borrowed = db.execute("SELECT COUNT(*) FROM issues WHERE user_id=? AND return_date IS NULL",
(uid,)).fetchone()[0]
    rows = with_fines(db.execute(ISSUE_QUERY + " WHERE i.user_id=?", (uid,)).fetchall())
    return render_template("dashboard.html", borrowed=borrowed,
                           fine=sum(r["current_fine"] for r in rows))



@app.route("/search")
@login_required()
def search():
    q = request.args.get("q", "").strip()
    like = f"%{q}%"
    books = get_db().execute(
        "SELECT * FROM books WHERE title LIKE ? OR author LIKE ? OR isbn LIKE ? OR category LIKE ?
ORDER BY title",
        (like, like, like, like)).fetchall()
    return render_template("search.html", books=books, q=q)



@app.route("/history")
@login_required("user")
def history():
    rows = get_db().execute(ISSUE_QUERY + " WHERE i.user_id=? ORDER BY i.id DESC",
(session["uid"],)).fetchall()
    return render_template("history.html", issues=with_fines(rows))



@app.route("/fines")
@login_required("user")
def my_fines():
    rows = with_fines(get_db().execute(ISSUE_QUERY + " WHERE i.user_id=? ORDER BY i.id DESC",
                                       (session["uid"],)).fetchall())
    rows = [r for r in rows if r["current_fine"] > 0]
    return render_template("my_fines.html", issues=rows,
                           total=sum(r["current_fine"] for r in rows), rate=FINE_PER_DAY)



@app.route("/materials")
@login_required()
def materials():
    year, sem = request.args.get("year", ""), request.args.get("semester", "")
    q = request.args.get("q", "").strip()
    sql, args = "SELECT * FROM materials WHERE (title LIKE ? OR subject LIKE ?)", [f"%{q}%", f"%{q}%"]
    if year:
        sql += " AND year=?"
        args.append(int(year))
    if sem:
        sql += " AND semester=?"
        args.append(int(sem))
    mats = get_db().execute(sql + " ORDER BY year, semester, title", args).fetchall()
    return render_template("materials.html", materials=mats, year=year, semester=sem, q=q)



@app.route("/download/<int:mid>")
@login_required()
def download(mid):
    m = get_db().execute("SELECT * FROM materials WHERE id=?", (mid,)).fetchone()
```

```
    if not m:
        flash("File not found.", "error")
        return redirect(url_for("materials"))
    return send_from_directory(UPLOADS, m["filename"], as_attachment=True)



@app.route("/profile", methods=["GET", "POST"])
@login_required()
def profile():
    db = get_db()
    if request.method == "POST":
        f = request.form
        db.execute("UPDATE users SET name=?, email=?, phone=? WHERE id=?",
                   (f["name"].strip(), f["email"].strip(), f["phone"].strip(), session["uid"]))
        if f.get("password"):
            db.execute("UPDATE users SET password=? WHERE id=?",
                       (generate_password_hash(f["password"]), session["uid"]))
        db.commit()
        session["name"] = f["name"].strip()
        flash("Profile updated.", "success")
        return redirect(url_for("profile"))
    user = db.execute("SELECT * FROM users WHERE id=?", (session["uid"],)).fetchone()
    return render_template("profile.html", user=user)



init_db()

if __name__ == "__main__":
    app.run(debug=True)
```

9.2 templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}City Library{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<header>
  <div class="bar">
    <a class="brand" href="{{ url_for('home') }}">&#128218; City Library</a>
    <nav>
      {% if session.role == 'admin' %}
        <a href="{{ url_for('admin_dashboard') }}">Dashboard</a>
        <a href="{{ url_for('admin_books') }}">Books</a>
        <a href="{{ url_for('issue_book') }}">Issue</a>
        <a href="{{ url_for('admin_issues') }}">Issues / Return</a>
        <a href="{{ url_for('admin_fines') }}">Fines</a>
        <a href="{{ url_for('admin_materials') }}">Material</a>
        <a href="{{ url_for('admin_messages') }}">Messages</a>
      {% elif session.role == 'user' %}
        <a href="{{ url_for('dashboard') }}">Home</a>
        <a href="{{ url_for('search') }}">Search Book</a>
        <a href="{{ url_for('history') }}">My Borrow History</a>
        <a href="{{ url_for('my_fines') }}">Check Fine</a>
        <a href="{{ url_for('materials') }}">Material</a>
        <a href="{{ url_for('profile') }}">My Profile</a>
      {% endif %}
      {% if session.uid %}
        <a href="{{ url_for('contact') }}">Contact</a>
        <a class="out" href="{{ url_for('logout') }}">Logout ({{ session.name }})</a>
      {% else %}
        <a href="{{ url_for('login') }}">Login</a>
        <a href="{{ url_for('register') }}">Register</a>
        <a href="{{ url_for('contact') }}">Contact</a>
      {% endif %}
    </nav>
  </div>
</header>
<main>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for cat, msg in messages %}<div class="flash {{ cat }}">{{ msg }}</div>{% endfor %}
  {% endwith %}
  {% block content %}{% endblock %}
</main>
```

```
<footer>City Library &middot; Internet Programming Lab &ndash; Experiment 1</footer>
</body>
</html>
```

9.3 templates/login.html

```
{% extends "base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>Login</h1>
  <form method="post">
    <label>Username</label><input name="username" required autofocus>
    <label>Password</label><input type="password" name="password" required>
    <p><button class="btn">Login</button></p>
  </form>
  <p class="muted">New student? <a href="{{ url_for('register') }}">Register here</a>.</p>
</div>
{% endblock %}
```

9.4 templates/register.html

```
{% extends "base.html" %}
{% block title %}Register{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>Student Registration</h1>
  <form method="post">
    <label>Full name</label><input name="name" required>
    <label>Username</label><input name="username" required>
    <label>Email</label><input type="email" name="email" required>
    <label>Phone</label><input name="phone" pattern="[0-9]{10}" title="10 digit number">
    <label>Password</label><input type="password" name="password" minlength="4" required>
    <p><button class="btn">Register</button></p>
  </form>
</div>
{% endblock %}
```

9.5 templates/contact.html

```
{% extends "base.html" %}
{% block title %}Contact{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>Contact Us</h1>
  <form method="post">
    <label>Name</label><input name="name" required>
    <label>Email</label><input type="email" name="email" required>
    <label>Message</label><textarea name="message" required></textarea>
    <p><button class="btn">Send</button></p>
  </form>
</div>
{% endblock %}
```

9.6 templates/admin_dashboard.html

```
{% extends "base.html" %}
{% block title %}Admin Dashboard{% endblock %}
{% block content %}
<h1>Librarian Dashboard</h1>
<div class="stats">
  <div class="stat"><b>{{ stats.books }}</b>Total book copies</div>
  <div class="stat"><b>{{ stats.available }}</b>Available now</div>
  <div class="stat"><b>{{ stats.issued }}</b>Currently issued</div>
  <div class="stat"><b>{{ stats.students }}</b>Registered students</div>
</div>
<div class="card">
  <a class="btn" href="{{ url_for('add_book') }}">+ Add Book</a>
  <a class="btn" href="{{ url_for('issue_book') }}">Issue Book</a>
  <a class="btn" href="{{ url_for('admin_issues', show='pending') }}">Pending Returns</a>
  <a class="btn" href="{{ url_for('admin_materials') }}">Upload Material</a>
</div>
{% endblock %}
```

9.7 templates/admin_books.html

```
{% extends "base.html" %}
{% block title %}Manage Books{% endblock %}
{% block content %}
```

```
<h1>Books</h1>
<div class="card">
  <form class="row" method="get">
    <div><label>Search</label><input name="q" value="{{ q }}" placeholder="Title, author, ISBN,
category"></div>
    <button class="btn">Search</button>
    <a class="btn green" href="{{ url_for('add_book') }}">+ Add Book</a>
  </form>
</div>
<div class="table-wrap"><table>
  <tr><th>ID</th><th>Title</th><th>Author</th><th>ISBN</th><th>Category</th><th>Available /
Total</th><th>Actions</th></tr>
  {% for b in books %}
  <tr>
    <td>{{ b.id }}</td><td>{{ b.title }}</td><td>{{ b.author }}</td><td>{{ b.isbn }}</td><td>{{
b.category }}</td>
    <td>{{ b.available_copies }} / {{ b.total_copies }}</td>
    <td>
      <a class="btn small" href="{{ url_for('edit_book', bid=b.id) }}">Edit</a>
      <form class="inline" method="post" action="{{ url_for('delete_book', bid=b.id) }}"
            onsubmit="return confirm('Delete this book?')"><button class="btn small
red">Delete</button></form>
    </td>
  </tr>
  {% else %}<tr><td colspan="7" class="muted">No books found.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.8 templates/book_form.html

```
{% extends "base.html" %}
{% block title %}{{ 'Edit' if book else 'Add' }} Book{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>{{ 'Update Book' if book else 'Add Book' }}</h1>
  <form method="post">
    <label>Title</label><input name="title" value="{{ book.title if book else '' }}" required>
    <label>Author</label><input name="author" value="{{ book.author if book else '' }}" required>
    <label>ISBN</label><input name="isbn" value="{{ book.isbn if book else '' }}">
    <label>Category</label><input name="category" value="{{ book.category if book else '' }}">
    <label>Total copies</label><input type="number" min="1" name="copies" value="{{ book.total_copies
if book else 1 }}" required>
    <p><button class="btn">Save</button> <a class="btn red" href="{{ url_for('admin_books')
}}">Cancel</a></p>
  </form>
</div>
{% endblock %}
```

9.9 templates/issue_book.html

```
{% extends "base.html" %}
{% block title %}Issue Book{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>Issue Book to Student</h1>
  <form method="post">
    <label>Student</label>
    <select name="user_id" required>
      <option value="">-- select student --</option>
      {% for s in students %}<option value="{{ s.id }}">{{ s.name }} ({{ s.username }})</option>{%
endfor %}
    </select>
    <label>Book</label>
    <select name="book_id" required>
      <option value="">-- select book --</option>
      {% for b in books %}<option value="{{ b.id }}">{{ b.title }} &ndash; {{ b.author }} ({{
b.available_copies }} left)</option>{% endfor %}
    </select>
    <p class="muted">Loan period is 14 days. Due date is set automatically.</p>
    <p><button class="btn">Issue Book</button></p>
  </form>
</div>
{% endblock %}
```

9.10 templates/admin_issues.html

```
{% extends "base.html" %}
{% block title %}Issues and Returns{% endblock %}
{% block content %}
<h1>Issued Books &amp;amp; Return Status</h1>
```

```
<p class="tabs">
  <a class="btn small {{ '' if show=='all' else 'green' }}" href="{{ url_for('admin_issues')
}}">All</a>
  <a class="btn small {{ '' if show=='pending' else 'green' }}" href="{{ url_for('admin_issues',
show='pending') }}">Not returned</a>
  <a class="btn small {{ '' if show=='returned' else 'green' }}" href="{{ url_for('admin_issues',
show='returned') }}">Returned</a>
</p>
<div class="table-wrap"><table>
  <tr><th>#</th><th>Student</th><th>Book</th><th>Issue date</th><th>Due date</th><th>Return
date</th><th>Fine (Rs.)</th><th>Status</th></tr>
  {% for i in issues %}
  <tr>
    <td>{{ i.id }}</td><td>{{ i.student }}</td><td>{{ i.title }}</td>
    <td>{{ i.issue_date }}</td><td>{{ i.due_date }}</td><td>{{ i.return_date or '-' }}</td>
    <td>{{ i.current_fine }}</td>
    <td>
      {% if i.return_date %}<span class="badge ok">Returned</span>
      {% else %}
        {% if i.current_fine > 0 %}<span class="badge bad">Overdue</span>{% else %}<span class="badge
warn">Issued</span>{% endif %}
        <form class="inline" method="post" action="{{ url_for('return_book', iid=i.id) }}"><button
class="btn small">Return</button></form>
      {% endif %}
    </td>
  </tr>
  {% else %}<tr><td colspan="8" class="muted">Nothing to show.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.11 templates/admin_fines.html

```
{% extends "base.html" %}
{% block title %}Fines{% endblock %}
{% block content %}
<h1>Fines</h1>
<p class="muted">Fine rule: Rs. {{ rate }} per day after the due date.</p>
<div class="table-wrap"><table>
  <tr><th>Student</th><th>Book</th><th>Due date</th><th>Return date</th><th>Fine (Rs.)</th></tr>
  {% for i in issues %}
  <tr><td>{{ i.student }}</td><td>{{ i.title }}</td><td>{{ i.due_date }}</td><td>{{ i.return_date or
'Not returned' }}</td><td>{{ i.current_fine }}</td></tr>
  {% else %}<tr><td colspan="5" class="muted">No fines.</td></tr>{% endfor %}
</table></div>
<p><b>Total fine: Rs. {{ total }}</b></p>
{% endblock %}
```

9.12 templates/admin_materials.html

```
{% extends "base.html" %}
{% block title %}Study Material{% endblock %}
{% block content %}
<h1>Study Material (Year / Semester wise)</h1>
<div class="card">
  <form method="post" enctype="multipart/form-data" class="row">
    <div><label>Title</label><input name="title" required></div>
    <div><label>Subject</label><input name="subject"></div>
    <div><label>Year</label>
      <select name="year" required>{% for y in range(1,5) %}<option>{{ y }}</option>{% endfor
%}</select></div>
    <div><label>Semester</label>
      <select name="semester" required>{% for s in range(1,9) %}<option>{{ s }}</option>{% endfor
%}</select></div>
    <div><label>File</label><input type="file" name="file" required></div>
    <button class="btn">Upload</button>
  </form>
</div>
<div class="table-wrap"><table>
  <tr><th>Year</th><th>Sem</th><th>Title</th><th>Subject</th><th>Uploaded</th><th>Actions</th></tr>
  {% for m in materials %}
  <tr>
    <td>{{ m.year }}</td><td>{{ m.semester }}</td><td>{{ m.title }}</td><td>{{ m.subject }}</td><td>{{
m.uploaded_on }}</td>
    <td>
      <a class="btn small" href="{{ url_for('download', mid=m.id) }}">View / Download</a>
      <form class="inline" method="post" action="{{ url_for('delete_material', mid=m.id) }}"
            onsubmit="return confirm('Delete this material?')"><button class="btn small
red">Delete</button></form>
    </td>
  </tr>
```

```
  {% else %}<tr><td colspan="6" class="muted">No material uploaded yet.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.13 templates/admin_messages.html

```
{% extends "base.html" %}
{% block title %}Messages{% endblock %}
{% block content %}
<h1>Contact Messages</h1>
<div class="table-wrap"><table>
  <tr><th>Date</th><th>Name</th><th>Email</th><th>Message</th></tr>
  {% for m in messages %}
  <tr><td>{{ m.sent_on }}</td><td>{{ m.name }}</td><td>{{ m.email }}</td><td>{{ m.message }}</td></tr>
  {% else %}<tr><td colspan="4" class="muted">No messages.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.14 templates/dashboard.html

```
{% extends "base.html" %}
{% block title %}My Library{% endblock %}
{% block content %}
<h1>Welcome, {{ session.name }}</h1>
<div class="stats">
  <div class="stat"><b>{{ borrowed }}</b>Books currently borrowed</div>
  <div class="stat"><b>Rs. {{ fine }}</b>Fine due</div>
</div>
<div class="card">
  <a class="btn" href="{{ url_for('search') }}">Search Book</a>
  <a class="btn" href="{{ url_for('history') }}">My Borrow History</a>
  <a class="btn" href="{{ url_for('materials') }}">Study Material</a>
</div>
{% endblock %}
```

9.15 templates/search.html

```
{% extends "base.html" %}
{% block title %}Search Books{% endblock %}
{% block content %}
<h1>Search Books</h1>
<div class="card">
  <form class="row" method="get">
    <div><label>Title / Author / ISBN / Category</label><input name="q" value="{{ q }}"
autofocus></div>
    <button class="btn">Search</button>
  </form>
</div>
<div class="table-wrap"><table>
  <tr><th>Title</th><th>Author</th><th>ISBN</th><th>Category</th><th>Availability</th></tr>
  {% for b in books %}
  <tr><td>{{ b.title }}</td><td>{{ b.author }}</td><td>{{ b.isbn }}</td><td>{{ b.category }}</td>
    <td>{% if b.available_copies > 0 %}<span class="badge ok">{{ b.available_copies }}
available</span>
        {% else %}<span class="badge bad">Not available</span>{% endif %}</td></tr>
  {% else %}<tr><td colspan="5" class="muted">No books found.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.16 templates/history.html

```
{% extends "base.html" %}
{% block title %}My Borrow History{% endblock %}
{% block content %}
<h1>My Borrow History</h1>
<div class="table-wrap"><table>
  <tr><th>Book</th><th>Author</th><th>Issue date</th><th>Expiry (due) date</th><th>Return
date</th><th>Fine (Rs.)</th><th>Status</th></tr>
  {% for i in issues %}
  <tr><td>{{ i.title }}</td><td>{{ i.author }}</td><td>{{ i.issue_date }}</td><td>{{ i.due_date
}}</td>
    <td>{{ i.return_date or '-' }}</td><td>{{ i.current_fine }}</td>
    <td>{% if i.return_date %}<span class="badge ok">Returned</span>
        {% elif i.current_fine > 0 %}<span class="badge bad">Overdue</span>
        {% else %}<span class="badge warn">Borrowed</span>{% endif %}</td></tr>
  {% else %}<tr><td colspan="7" class="muted">You have not borrowed any books yet.</td></tr>{% endfor
%}
</table></div>
{% endblock %}
```

9.17 templates/my_fines.html

```
{% extends "base.html" %}
{% block title %}My Fines{% endblock %}
{% block content %}
<h1>Check Fine</h1>
<p class="muted">Fine rule: Rs. {{ rate }} per day after the due date.</p>
<div class="table-wrap"><table>
  <tr><th>Book</th><th>Due date</th><th>Return date</th><th>Fine (Rs.)</th></tr>
  {% for i in issues %}
  <tr><td>{{ i.title }}</td><td>{{ i.due_date }}</td><td>{{ i.return_date or 'Not returned yet'
}}</td><td>{{ i.current_fine }}</td></tr>
  {% else %}<tr><td colspan="4" class="muted">You have no fines.</td></tr>{% endfor %}
</table></div>
<p><b>Total fine: Rs. {{ total }}</b></p>
{% endblock %}
```

9.18 templates/materials.html

```
{% extends "base.html" %}
{% block title %}Study Material{% endblock %}
{% block content %}
<h1>Search Study Material</h1>
<div class="card">
  <form class="row" method="get">
    <div><label>Keyword</label><input name="q" value="{{ q }}"></div>
    <div><label>Year</label>
      <select name="year"><option value="">All</option>
        {% for y in range(1,5) %}<option value="{{ y }}" {{ 'selected' if year == y|string }}>{{ y
}}</option>{% endfor %}</select></div>
    <div><label>Semester</label>
      <select name="semester"><option value="">All</option>
        {% for s in range(1,9) %}<option value="{{ s }}" {{ 'selected' if semester == s|string }}>{{ s
}}</option>{% endfor %}</select></div>
    <button class="btn">Search</button>
  </form>
</div>
<div class="table-wrap"><table>
  <tr><th>Year</th><th>Sem</th><th>Title</th><th>Subject</th><th>Download</th></tr>
  {% for m in materials %}
  <tr><td>{{ m.year }}</td><td>{{ m.semester }}</td><td>{{ m.title }}</td><td>{{ m.subject }}</td>
    <td><a class="btn small" href="{{ url_for('download', mid=m.id) }}">Download</a></td></tr>
  {% else %}<tr><td colspan="5" class="muted">No material found.</td></tr>{% endfor %}
</table></div>
{% endblock %}
```

9.19 templates/profile.html

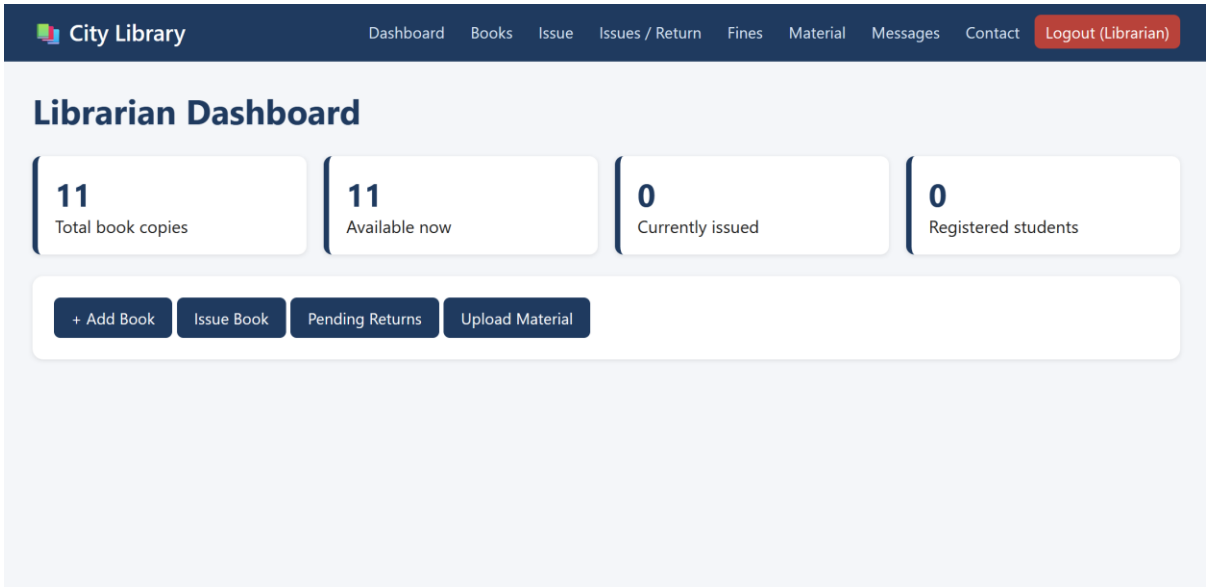
```
{% extends "base.html" %}
{% block title %}My Profile{% endblock %}
{% block content %}
<div class="card narrow">
  <h1>My Profile</h1>
  <form method="post">
    <label>Username</label><input value="{{ user.username }}" disabled>
    <label>Full name</label><input name="name" value="{{ user.name }}" required>
    <label>Email</label><input type="email" name="email" value="{{ user.email or '' }}">
    <label>Phone</label><input name="phone" value="{{ user.phone or '' }}">
    <label>New password (leave blank to keep current)</label><input type="password" name="password">
    <p><button class="btn">Update Profile</button></p>
  </form>
</div>
{% endblock %}
```

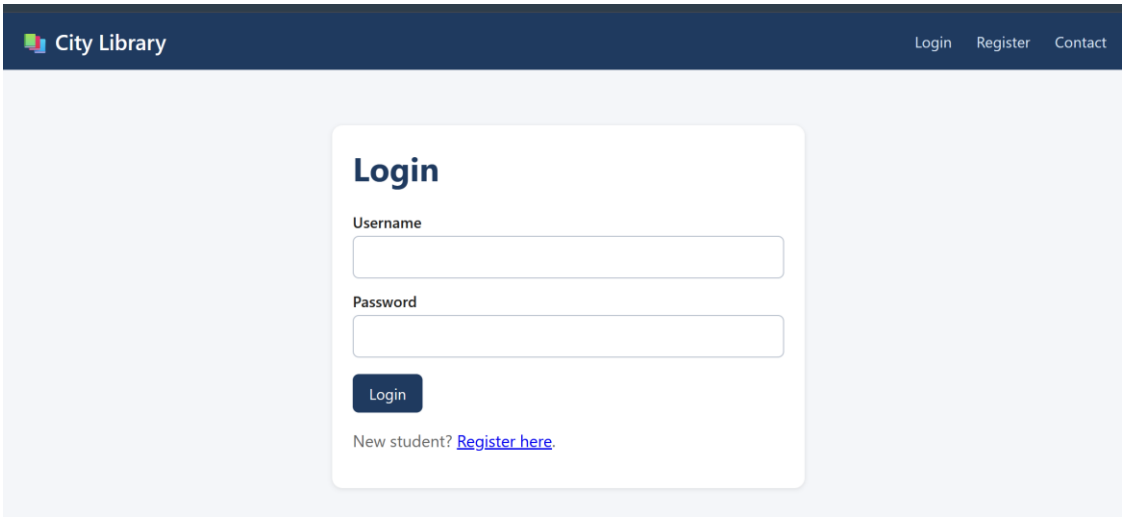
9.20 static/style.css

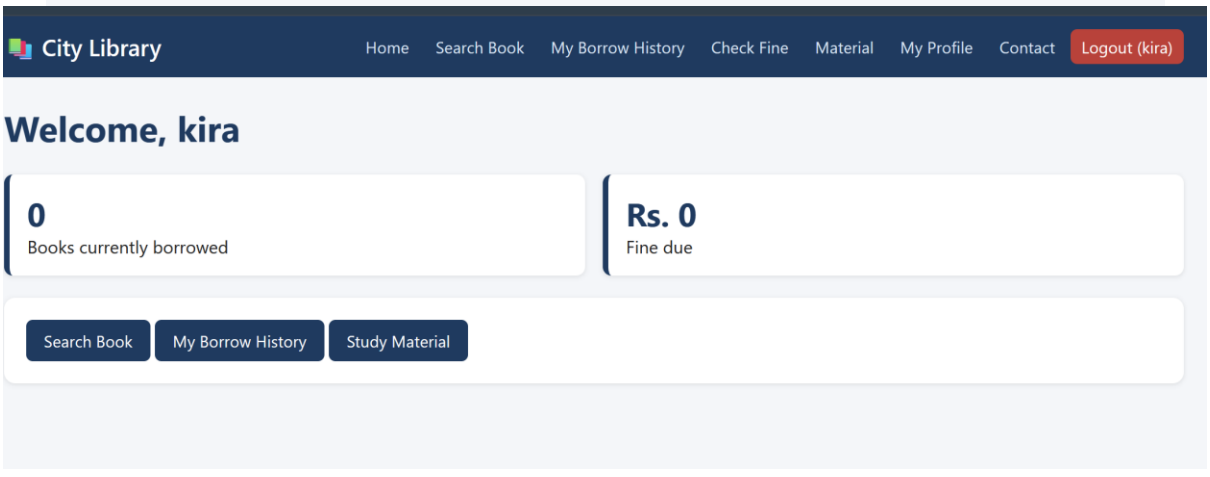
```
* { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", Arial, sans-serif; background: #f4f6f9; color: #222; min-
height: 100vh; display: flex; flex-direction: column; }
header { background: #1f3a5f; color: #fff; }
.bar { max-width: 1100px; margin: auto; padding: 12px 16px; display: flex; flex-wrap: wrap; align-
items: center; gap: 12px; justify-content: space-between; }
.brand { color: #fff; font-size: 1.3rem; font-weight: 600; text-decoration: none; }
nav { display: flex; flex-wrap: wrap; gap: 4px; }
nav a { color: #dce6f5; text-decoration: none; padding: 6px 10px; border-radius: 6px; font-size:
.92rem; }
nav a:hover { background: #2f5287; color: #fff; }
nav a.out { background: #b8423a; color: #fff; }
main { flex: 1; width: 100%; max-width: 1100px; margin: 24px auto; padding: 0 16px; }
footer { text-align: center; padding: 14px; font-size: .85rem; color: #666; }
```

```
h1 { margin-top: 0; color: #1f3a5f; }
.card { background: #fff; border-radius: 10px; padding: 20px; box-shadow: 0 1px 4px rgba(0,0,0,.1);
margin-bottom: 20px; }
.narrow { max-width: 460px; margin: 30px auto; }
.stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px;
margin-bottom: 20px; }
.stat { background: #fff; border-left: 5px solid #1f3a5f; border-radius: 8px; padding: 16px; box-
shadow: 0 1px 4px rgba(0,0,0,.1); }
.stat b { display: block; font-size: 1.8rem; color: #1f3a5f; }
label { display: block; margin: 12px 0 4px; font-weight: 600; font-size: .92rem; }
input, select, textarea { width: 100%; padding: 9px 10px; border: 1px solid #bbc3cf; border-radius:
6px; font: inherit; }
textarea { min-height: 110px; }
.btn { display: inline-block; background: #1f3a5f; color: #fff; border: 0; padding: 9px 16px; border-
radius: 6px; cursor: pointer; text-decoration: none; font: inherit; font-size: .92rem; }
.btn:hover { background: #2f5287; }
.btn.red { background: #b8423a; } .btn.green { background: #2e8540; } .btn.small { padding: 5px 10px;
font-size: .82rem; }
form.inline { display: inline; }
.row { display: flex; flex-wrap: wrap; gap: 10px; align-items: end; }
.row > * { flex: 1; min-width: 140px; } .row .btn { flex: 0 0 auto; }
table { width: 100%; border-collapse: collapse; background: #fff; border-radius: 8px; overflow:
hidden; box-shadow: 0 1px 4px rgba(0,0,0,.1); }
th, td { padding: 10px 12px; text-align: left; border-bottom: 1px solid #e6e9ef; font-size: .92rem; }
th { background: #e9eef6; color: #1f3a5f; }
.table-wrap { overflow-x: auto; }
.flash { padding: 10px 14px; border-radius: 6px; margin-bottom: 14px; }
.flash.success { background: #def3e2; color: #1d5a2a; } .flash.error { background: #fbe0de; color:
#8a2822; }
.badge { padding: 2px 8px; border-radius: 10px; font-size: .78rem; color: #fff; }
.badge.ok { background: #2e8540; } .badge.warn { background: #d98e04; } .badge.bad { background:
#b8423a; }
.muted { color: #666; }
.tabs a { margin-right: 8px; }
```

10. Output


City Library
Dashboard
Books
Issue
Issues / Return
Fines
Material
Messages
Contact
Logout (Librarian)
Librarian Dashboard
11
Total book copies
11
Available now
0
Currently issued
0
Registered students
+ Add Book
Issue Book
Pending Returns
Upload Material


City Library
Login
Register
Contact
Login
Username
Password
Login
New student? Register here.


City Library
Home
Search Book
My Borrow History
Check Fine
Material
My Profile
Contact
Logout (kira)
Welcome, kira
0
Books currently borrowed
Rs. 0
Fine due
Search Book
My Borrow History
Study Material